

Apache Spark: MLlib

Por Prof. Oscar H. Mondragón

1. Objetivos

- Explorar las funcionalidades ofrecidas por MLlib
- Probar una implementación básica la librería de LinearRegression de MLlib

2. Herramientas a utilizar

- Vagrant
- VirtualBox
- Apache Spark

3. Introducción

En esta práctica, exploraremos las funcionalidades ofrecidas por la librería MLlib de machine learning de Apache Spark y aprenderemos a implementar un modelo básico de regresión lineal con la función **LinearRegression**.

Para esto, trabajaremos con un conjunto de datos de alquiler de bicicletas, el cual cargaremos y manipularemos utilizando Spark. Además, utilizaremos la librería **VectorAssembler** para convertir las características del conjunto de datos a un vector denso, lo cual nos permitirá alimentar el modelo de regresión lineal. Al final de esta práctica, tendrás una comprensión de las capacidades de MLlib y sabrás cómo implementar un modelo de regresión lineal básico utilizando esta librería.

4. Desarrollo de la práctica

4.1. Configuración de Vagrant

Esta práctica la desarrollaremos usando boxes de Ubuntu 22.04 en Vagrant. El Vagrantfile que usaremos es el siguiente (el mismo con el que venimos trabajando):

```
# -*- mode: ruby -*-  
# vi: set ft=ruby :
```

```
Vagrant.configure("2") do |config|  
  
  if Vagrant.has_plugin? "vagrant-vbguest"  
    config.vbguest.no_install = true  
    config.vbguest.auto_update = false  
    config.vbguest.no_remote = true  
  end  
end
```

```

end

config.vm.define :clienteUbuntu do |clienteUbuntu|
  clienteUbuntu.vm.box = "bento/ubuntu-22.04"
  clienteUbuntu.vm.network :private_network, ip: "192.168.100.2"
  clienteUbuntu.vm.hostname = "clienteUbuntu"
end

config.vm.define :servidorUbuntu do |servidorUbuntu|
  servidorUbuntu.vm.box = "bento/ubuntu-22.04"
  servidorUbuntu.vm.network :private_network, ip: "192.168.100.3"
  servidorUbuntu.vm.hostname = "servidorUbuntu"
end
end

```

4.2. Preparación de la Práctica

En prácticas anteriores, configuramos un cluster Spark de un solo nodo. Lo primero que haremos es Iniciar el master:

```

vagrant@servidorUbuntu:~$ cd labSpark/spark-3.5.0-bin-hadoop3/sbin/
vagrant@servidorUbuntu:~/labSpark/spark-3.5.0-bin-hadoop3/sbin$ ./start-
master.sh
starting org.apache.spark.deploy.master.Master, logging to
/home/vagrant/labSpark/spark-3.5.0-bin-hadoop3/logs/spark-vagrant-
org.apache.spark.deploy.master.Master-1-servidorUbuntu.out

```

Seguidamente, lanzaremos un worker:

```

vagrant@servidorUbuntu:~/labSpark/spark-3.5.0-bin-hadoop3/sbin$ ./start-
worker.sh spark://192.168.100.3:7077
starting org.apache.spark.deploy.worker.Worker, logging to
/home/vagrant/labSpark/spark-3.5.0-bin-hadoop3/logs/spark-vagrant-
org.apache.spark.deploy.worker.Worker-1-servidorUbuntu.out

```

Puede verificar que el master y el worker están corriendo correctamente abriendo <http://192.168.100.3:8080/> en el browser.

Spark Master at spark://192.168.100.3:7077

URL: spark://192.168.100.3:7077
 Alive Workers: 1
 Cores in use: 2 Total, 0 Used
 Memory in use: 1024.0 MiB Total, 0.0 B Used
 Resources in use:
 Applications: 0 Running, 0 Completed
 Drivers: 0 Running, 0 Completed
 Status: ALIVE

▼ Workers (1)

Worker Id	Address	State	Cores	Memory	Resources
worker-20230216152735-192.168.100.3-42131	192.168.100.3:42131	ALIVE	2 (0 Used)	1024.0 MiB (0.0 B Used)	

▼ Running Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

▼ Completed Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

4.3. Explorando Spark MLlib

Clone el repositorio github disponible en:
<https://github.com/omondragon/bikeRentals>

```
vagrant@servidorUbuntu:~/labSpark$ git clone
https://github.com/omondragon/bikeRentals
```

Este repositorio contiene los siguientes archivos:

rentals.py

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, IntegerType, FloatType

# Crear sesión de Spark
spark = SparkSession.builder.appName("Bike Rentals Prediction").getOrCreate()

# Definir el esquema de datos
```

```

schema = StructType([
    StructField("hour", IntegerType(), True),
    StructField("temperature", FloatType(), True),
    StructField("visitors", IntegerType(), True),
    StructField("rentals", IntegerType(), True)
])

# Cargar los datos de entrenamiento
data = spark.read.format("csv").option("header",
"true").schema(schema).load("/home/vagrant/labSpark/bikeRentals/bike_rentals.csv")

# Convertir las características a un vector denso
assembler = VectorAssembler(inputCols=["hour", "temperature", "visitors"],
outputCol="features")
data = assembler.transform(data)

# Dividir los datos en conjunto de entrenamiento y conjunto de prueba
train_data, test_data = data.randomSplit([0.7, 0.3])

# Crear modelo de regresión lineal y entrenarlo con los datos de entrenamiento
lr = LinearRegression(featuresCol="features", labelCol="rentals")
model = lr.fit(train_data)

# Hacer predicciones en el conjunto de prueba
predictions = model.transform(test_data)

# Mostrar las predicciones
predictions.select("rentals", "prediction").show()

```

Este script de Python que utiliza la librería MLlib de Apache Spark para entrenar un modelo de regresión lineal sobre un conjunto de datos de alquiler de bicicletas. El script carga los datos del archivo **bike_rentals.csv**, los transforma utilizando la librería **VectorAssembler**, los divide en un conjunto de entrenamiento y un conjunto de prueba, y finalmente entrena el modelo y realiza predicciones sobre el conjunto de prueba.

Por otra parte, este repositorio contiene el archivo **bike_rentals.csv**. Este archivo es un conjunto de datos de alquiler de bicicletas, que contiene información sobre la hora del día, la temperatura, el número de visitantes y el número de alquileres de bicicletas. Este conjunto de datos se utiliza en el script **rentals.py** para entrenar y evaluar el modelo de regresión lineal.

Es importante destacar que el archivo **rentals.py** asume que el archivo **bike_rentals.csv** se encuentra en la ruta **/home/vagrant/labSpark/bikeRentals/bike_rentals.csv** del sistema de archivos del cluster de Spark en el que se está ejecutando el script. Por lo tanto, es importante asegurarse de que este archivo esté ubicado en esa ruta antes de ejecutar el script.

Para instalar numpy ejecute.

```
sudo apt-get install python3-pip
pip3 install numpy
```

Ejecute la aplicación

Ejecute la aplicación y verifique el resultado:

```
vagrant@servidorUbuntu:~/labSpark/spark-3.5.0-bin-hadoop3/bin$ ./spark-submit -
-master spark://192.168.100.3:7077
/home/vagrant/labSpark/bikeRentals/rentals.py
23/02/28 15:07:25 INFO SparkContext: Running Spark version 3.5.0
23/02/28 15:07:25 WARN NativeCodeLoader: Unable to load native-hadoop library
for your platform... using builtin-java classes where applicable
...
23/02/28 15:07:47 INFO CodeGenerator: Code generated in 13.288758 ms
+-----+-----+
|rentals|      prediction|
+-----+-----+
|    12|11.146489825343327|
|     6| 6.040453559271601|
|     4| 1.099507833108083|
|    15|13.685602473649258|
|    30| 31.60310710334376|
|    35| 34.60831030742548|
|    30| 28.82533308885364|
|     6| 3.965240939444124|
+-----+-----+

23/02/28 15:07:47 INFO SparkContext: Invoking stop() from shutdown hook
23/02/28 15:07:47 INFO SparkUI: Stopped Spark web UI at
http://192.168.100.3:4040
...
```

5. Ejercicio

1. Modifique **bike_rentals.csv** agregando más datos y compruebe el funcionamiento del modelo.
2. Cuando se trabajan modelos de machine learning, es importante evaluar la calidad del modelo. En este punto se solicita la implementación de una métrica de evaluación del modelo, como el cálculo del error cuadrático medio (RMSE) o el coeficiente de determinación (R-cuadrado), para

evaluar el rendimiento del modelo de regresión lineal. Esto ayuda a comprender la calidad del modelo y su capacidad para hacer predicciones precisas.

HINT:

Para este punto puede usar la librería `pyspark.ml.evaluation`

Explique el resultado obtenido.

6. Entregables y Evaluación

- Sustentación de la práctica

7. Bibliografía

- Machine Learning Library (MLlib) Guide. <https://spark.apache.org/docs/latest/ml-guide.html>
- Tutorial PySpark Interactivo. <https://andfanilo.github.io/pyspark-interactive-lecture/#/>